

Cache Aside e Performance de APIs

Arquitetura de Software — 5º período | PUC Minas

Rails 7.2 · PostgreSQL 16 · Redis 7 · Docker Compose · k6 · Grafana

Slide 1 — Problema

Contexto: API acadêmica com consultas repetitivas que sobrecarregam o banco.

- Toda requisição de leitura vai ao PostgreSQL — sem atalho
- Dados de catálogo (produtos, clientes, pedidos) mudam pouco, mas são lidos com alta frequência
- Sob carga, o banco vira gargalo: **latência dispara, throughput cai**

Solução proposta: padrão **Cache Aside** com Redis como camada intermediária

Slide 2 — O Padrão Cache Aside

LEITURA

1. Consulta o Redis
 - HIT: retorna direto (banco não é tocado)
 - MISS: consulta o banco → grava no Redis → retorna

ESCRITA (create / update / delete)

1. Persiste no banco
2. Invalida a chave no Redis (apaga)
 - próxima leitura vai ao banco e renova o cache

Por que invalidar em vez de atualizar o cache?

Invalidar é mais simples e elimina a janela de cache inconsistente — sem precisar garantir atomicidade na re-serialização do objeto.

Slide 3 — Arquitetura da Solução

```
Cliente HTTP
  ↓
Rails 7.2 (API-only)
  ├── CacheService.fetch(key) { ... }
  │   ├── [HIT] Redis ← resposta em memória, banco não tocado
  │   └── [MISS] PostgreSQL → grava no Redis → resposta
  ├── CacheService.invalidate(*keys)
  │   └── apaga chaves após create / update / delete
  └── CacheStats → GET /stats → hits · misses · invalidations

Observabilidade
  Rails logs → Promtail → Loki → Grafana (:3001)
  Relatório k6 → http://localhost:8080/report/
```

Toggle: variável de ambiente `CACHE_ENABLED=true|false` — mesma imagem, dois cenários

Slide 4 — Metodologia dos Testes

Ferramenta: k6 — 200 usuários virtuais simultâneos · 40 req/s · ~220 s de duração

ETAPA	REQUISIÇÃO	DETALHE
Catálogo	GET /products	Lista todos os produtos
Produto	GET /products/:id	Detalha 1–2 produtos
Pedidos	GET /orders	JOIN pesado (pedidos + itens)
Compra	POST /orders (25%)	Cria pedido → invalida cache

Dataset: 300 clientes · 500 produtos · 2.000 pedidos — mesma seed nos dois cenários

Dois cenários isolados:

- cache-off → CACHE_ENABLED=false — toda requisição vai ao banco
- cache-on → CACHE_ENABLED=true — padrão cache-aside ativo · Redis zerado antes do run

Slide 5 — Resultados: Latência

MÉTRICA	SEM CACHE	COM CACHE	REDUÇÃO
Latência média	12.379 ms	5.471 ms	55,8% ↓
Latência p90	16.686 ms	7.461 ms	55,3% ↓
Latência p95	17.290 ms	7.713 ms	55,4% ↓
Latência p99	17.889 ms	8.170 ms	54,3% ↓
Latência máx.	18.786 ms	9.010 ms	52,0% ↓

O cache reduziu **~55% o tempo de resposta** em todos os percentis, sob carga de 200 usuários simultâneos.

Slide 6 — Resultados: Throughput

MÉTRICA	SEM CACHE	COM CACHE	VARIAÇÃO
Requisições totais	2.982	6.044	+102,7% ↑
Throughput (req/s)	13,54	28,14	+107,8% ↑
Iterações concluídas	728	1.617	+122,1% ↑
Iterações dropadas	5.450	4.657	14,5% ↓

Com cache, o servidor entregou **mais que o dobro de requisições** no mesmo tempo e completou **2,2× mais jornadas** de cliente.

Slide 7 — Cache Hit, Miss e Invalidação

Medido durante o cenário `cache-on` com 200 usuários virtuais:

CONTADOR	VALOR
Cache HITs	420
Cache MISSes	291
Hit Rate	59,1%
Invalidações	49

Consultas ao banco:

CENÁRIO	CONSULTAS	DIFERENÇA
Sem cache	1.422	base
Com cache	804	43,5% ↓

As 49 invalidações correspondem às compras (`POST /orders`) — cada compra apaga `orders:all` e renova o cache na próxima leitura.

Slide 8 — Trade-offs Arquiteturais

DIMENSÃO	SEM CACHE	COM CACHE
Latência	Alta — banco toda vez	Baixa nos HITs (~55% menos)
Throughput	Limitado pelo banco	~2× maior
Consistência	Forte	Eventual (janela mínima: 1 req)
Complexidade	Simples	Maior — invalidação + TTL
Resiliência	Ponto único: banco	Redis offline → fallback ao banco

Estratégia de consistência — `invalidate-on-write` :

Escrita → banco persistido → chave **apagada** (não atualizada) → próxima leitura renova

TTL = 300 s como rede de segurança:

Se a invalidação falhar, a chave expira em 5 min — sem intervenção manual.

Slide 9 — Conclusão

OBJETIVO DO ENUNCIADO	RESULTADO MEDIDO
Reduzir latência	55% menor ↓ em todos os percentis
Diminuir carga no banco	43,5% menos consultas ↓
Melhorar desempenho	+108% throughput ↑
Demonstrar hit / miss	59,1% hit rate · 49 invalidações
Discutir consistência	Eventual · invalidate-on-write · TTL 300 s

Lições:

- 1. Cache-aside dá controle total — e responsabilidade total — para a aplicação
- 2. Invalidar é mais seguro que atualizar: elimina dado inconsistente ativo no Redis
- 3. TTL não é apenas otimização de memória — é rede de segurança para falhas de invalidação
- 4. 59% de hit rate já entrega ganho expressivo; sistemas reais com dados estáveis atingem 80–95%